
Kubernetes Admission Controller
or how Captain Hook learned Cloud Security

Sooo we want …

… to deploy our Workload
in Kubernetes




Lars
IT-Security
YOU'RE GOING TO DEPLOY WHAT?

THE ISSUE

- Dynamic nature of Cloud Native Patterns
- (Security) Tools **not** adopted for Cloud Native Patterns
- **Not** secure by default
- Clusters are **not** immutable
- Policy becomes arcane tribal knowledge and **not** defined in code

Traditional Solution Approaches

- Manual Interventions
- Restrict users from creating objects e.g. DaemonSets



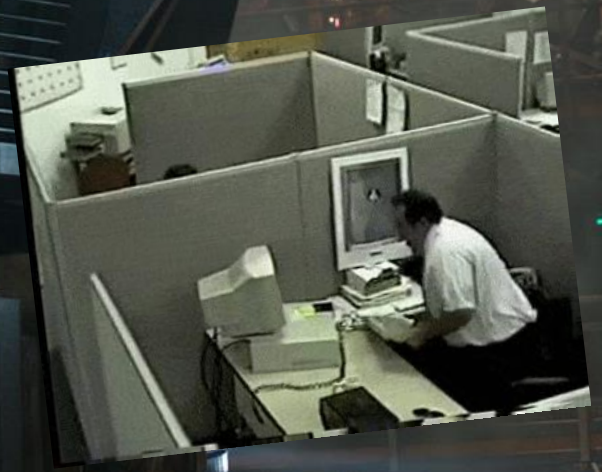
The resulting (security) architecture



The resulting (security) architecture

leads to frustration



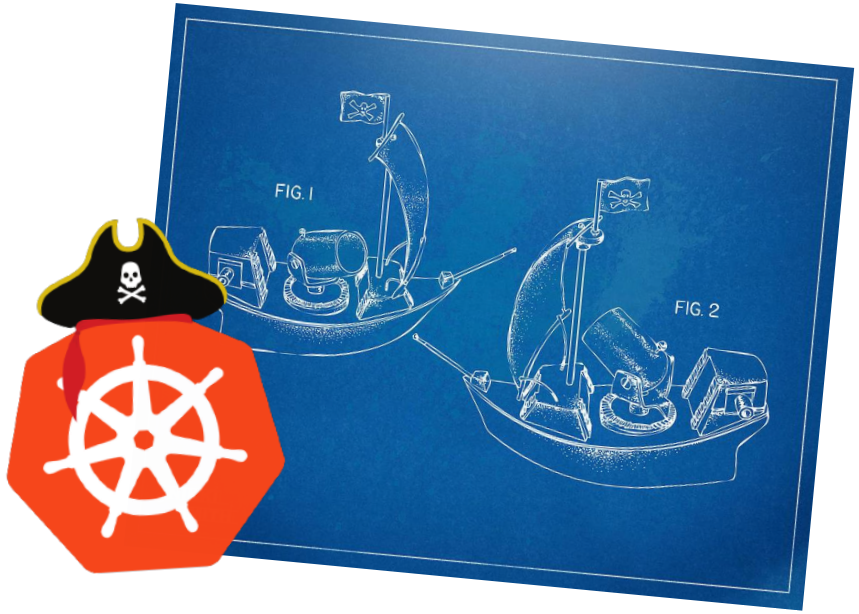


But Kubernetes without security compliance ...

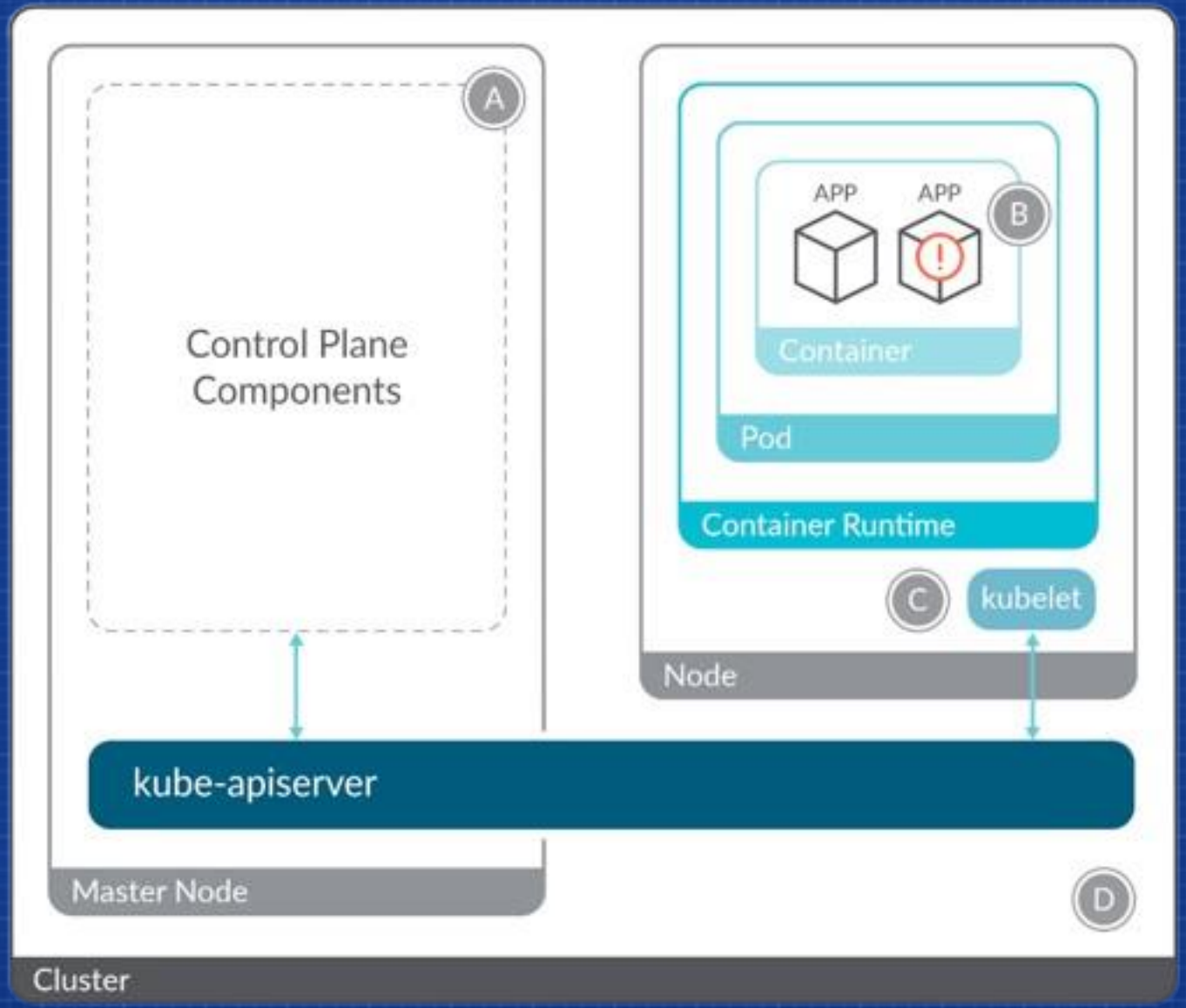


What‘s the Attack Surface ?

… of a Kubernetes Cluster



KUBERNETES ATTACK SURFACE

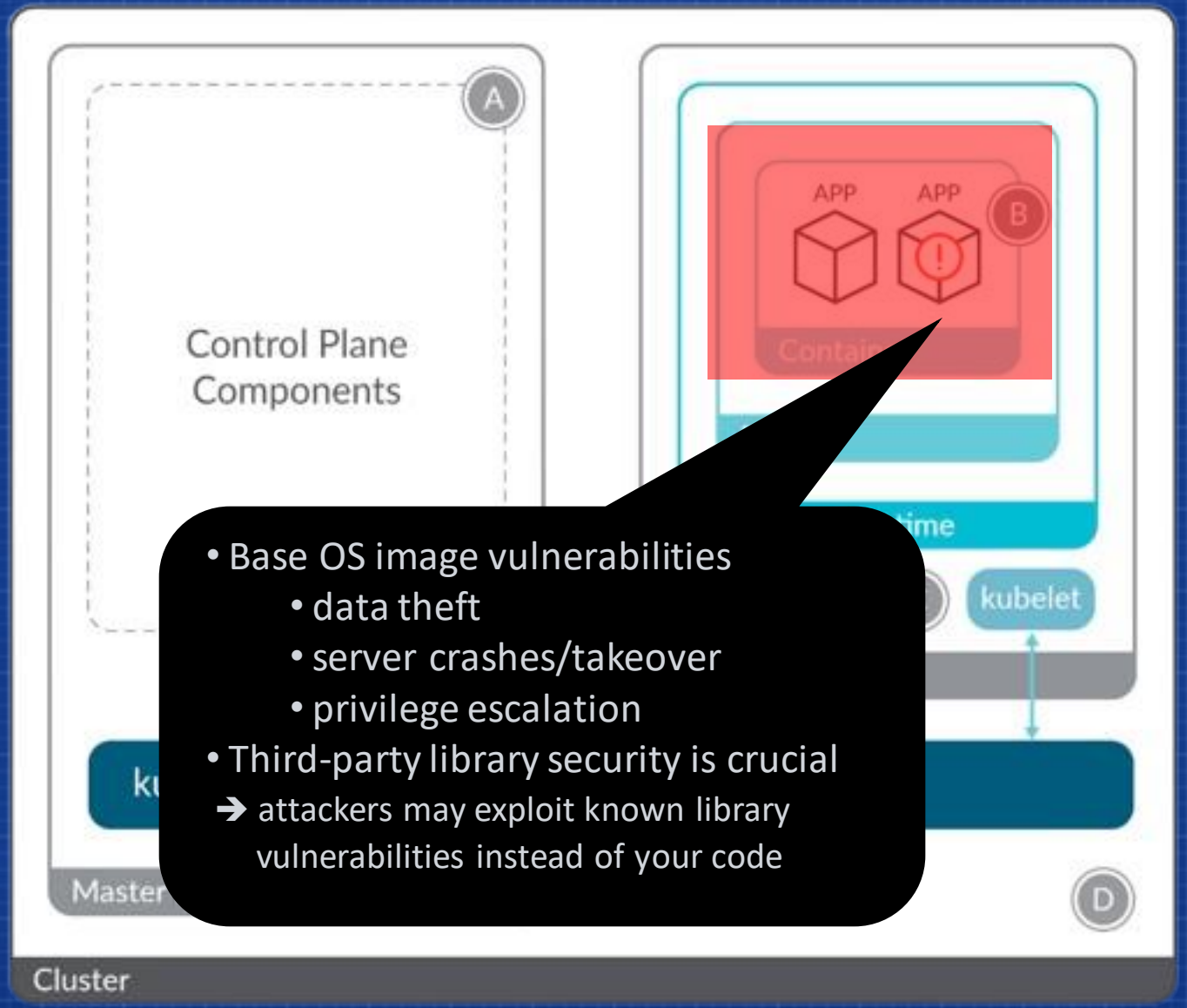


Source: Sysdig – k8s Security guide

Elements to protect

- A — Access via Kubernetes API Proxy etcd API
- B — Exploit vulnerability in apps or 3rd party libraries
- C — Access via API
- D — Access to the servers or virtual machines

KUBERNETES ATTACK SURFACE

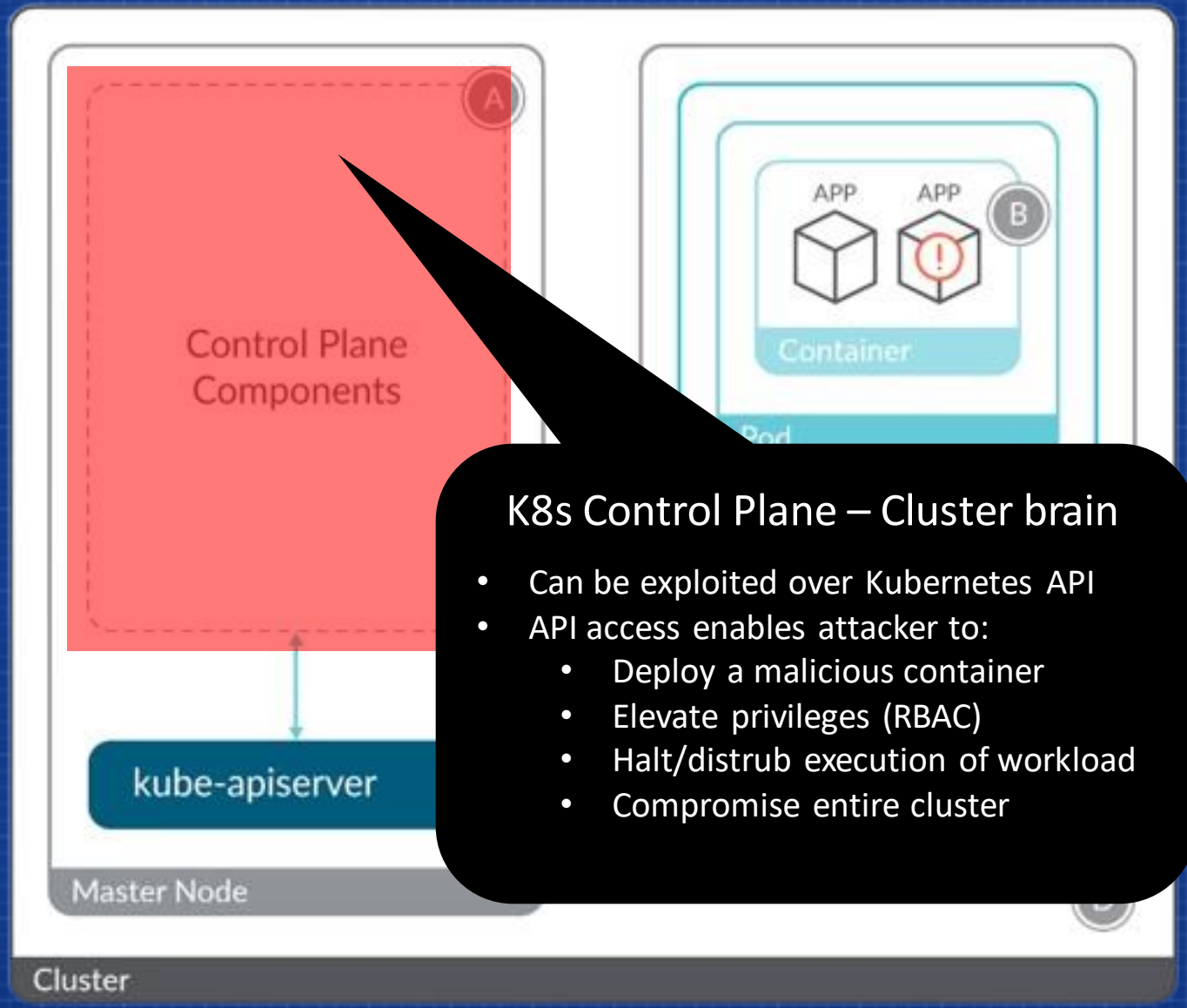


Elements to protect

- **A** Access via Kubernetes API Proxy etcd API
- **B** Exploit vulnerability in apps or 3rd party libraries
- **C** Access via API
- **D** Access to the servers or virtual machines

Source: Sysdig – k8s Security guide

KUBERNETES ATTACK SURFACE



K8s Control Plane – Cluster brain

- Can be exploited over Kubernetes API
- API access enables attacker to:
  - Deploy a malicious container
  - Elevate privileges (RBAC)
  - Halt/distrub execution of workload
  - Compromise entire cluster

Elements to protect

- A — Access via Kubernetes API Proxy etcd API
- B — Exploit vulnerability in apps or 3rd party libraries
- C — Access via API
- D — Access to the servers or virtual machines

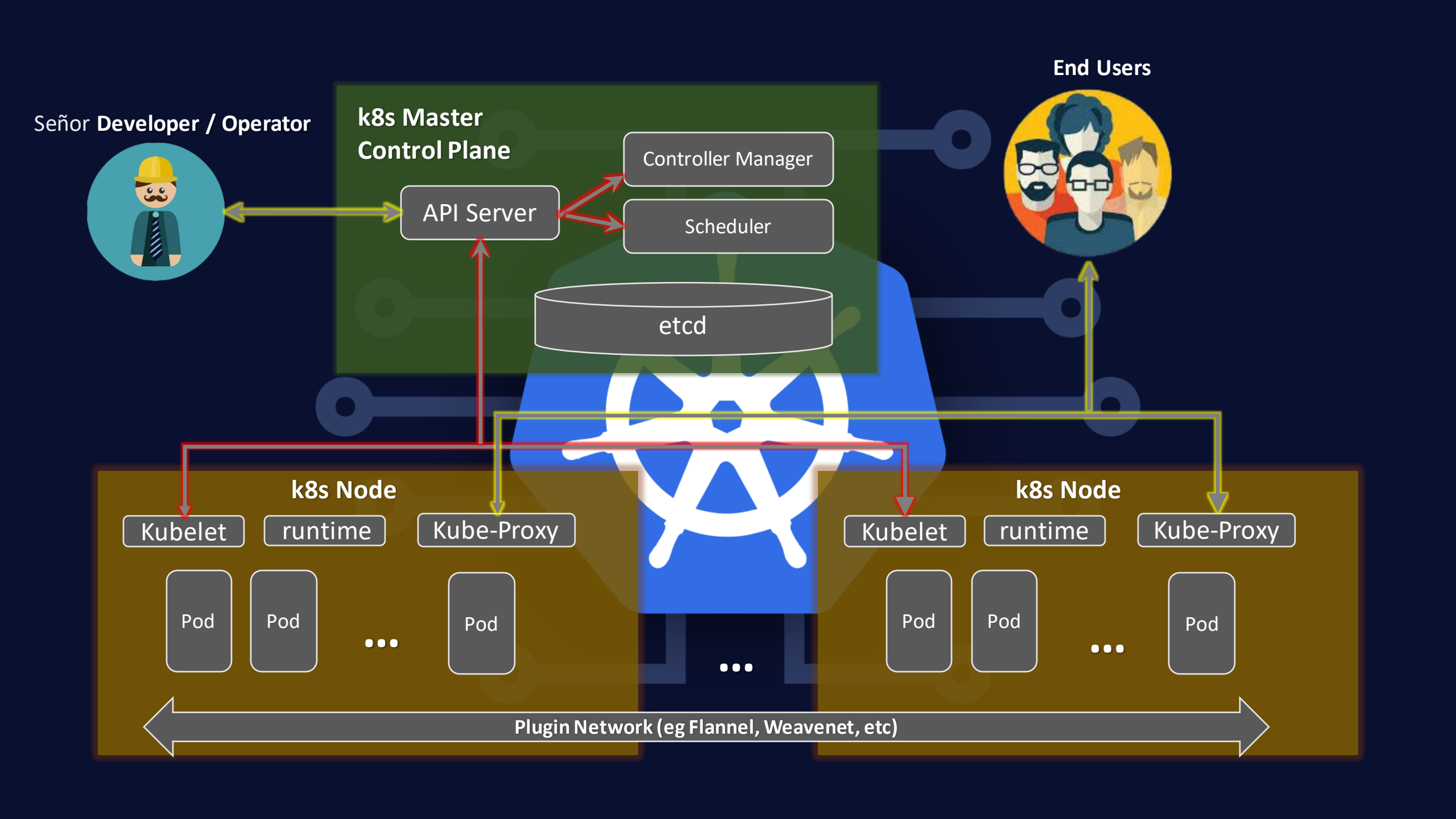
Source: Sysdig – k8s Security guide

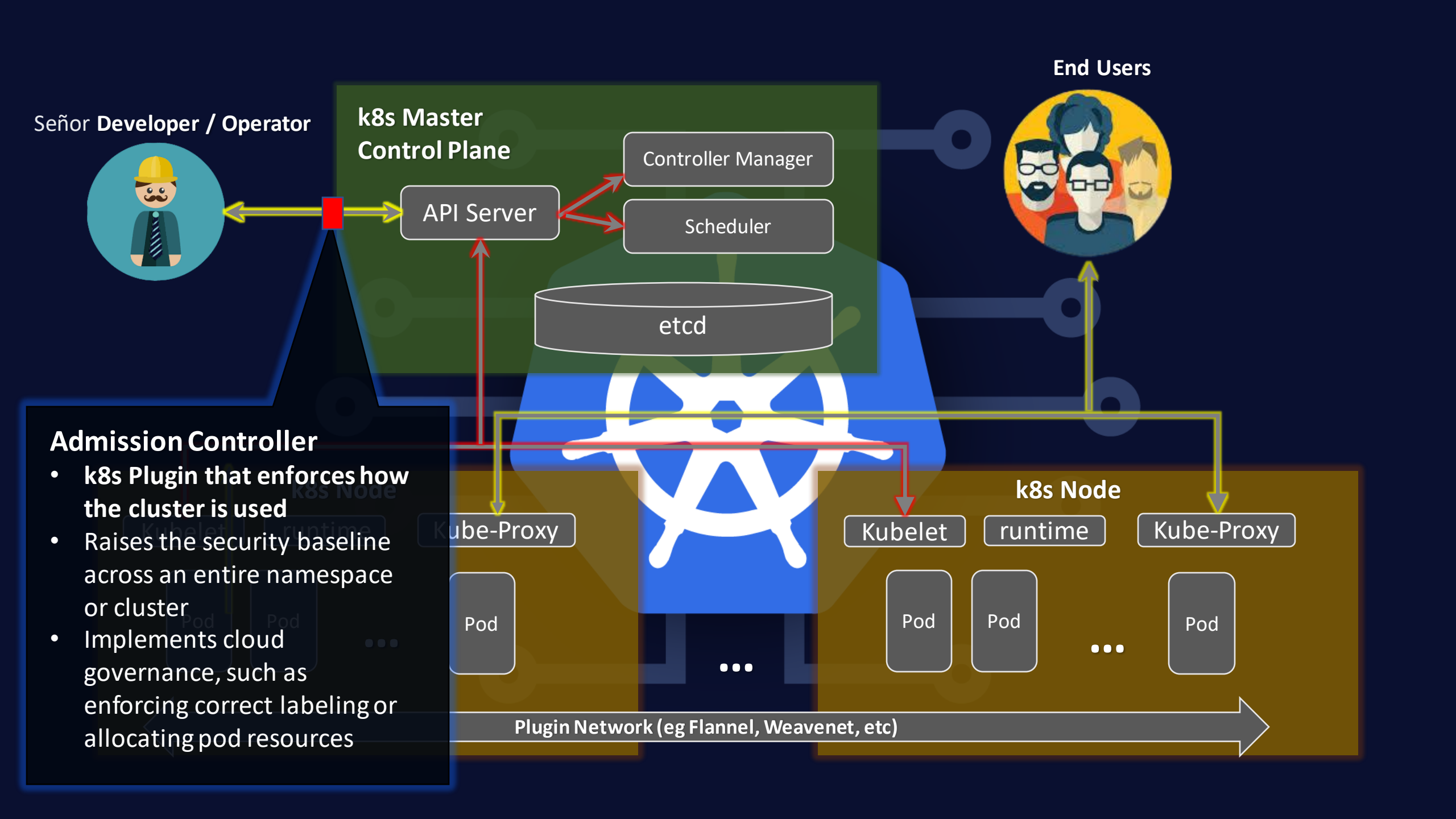
What is actually ?

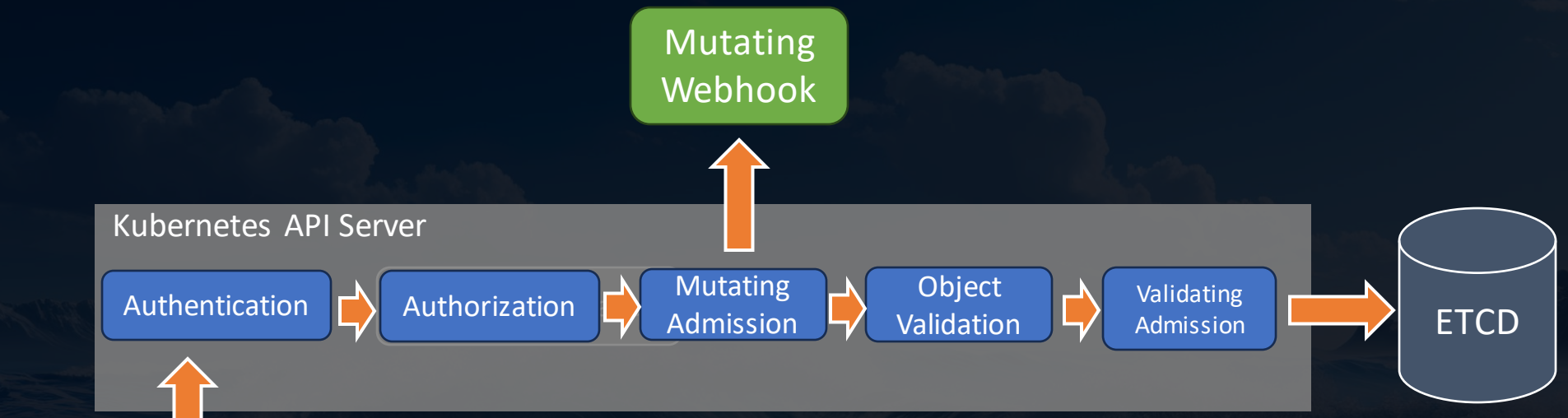
..an

Admission Controller


An admission controller is a piece of code that intercepts requests to the Kubernetes API server prior to persistence of the object, but after the request is authenticated and authorized.
- Kubernetes.io


End Users
Señor Developer / Operator
k8s Master
Control Plane
Controller Manager
API Server
Scheduler
etcd
k8s Node
Kubelet
runtime
Kube-Proxy
Pod
Pod
Pod
k8s Node
Kubelet
runtime
Kube-Proxy
Pod
Pod
Pod
Plugin Network (eg Flannel, Weavenet, etc)


End Users
Señor Developer / Operator
k8s Master
Control Plane
Controller Manager
API Server
Scheduler
etcd
Admission Controller
• k8s Plugin that enforces how the cluster is used
• Raises the security baseline across an entire namespace or cluster
• Implements cloud governance, such as enforcing correct labeling or allocating pod resources
k8s Node
Kube-Proxy
Kubelet
runtime
Kube-Proxy
Pod
Pod
Pod
Pod
Pod
Plugin Network (eg Flannel, Weavenet, etc)

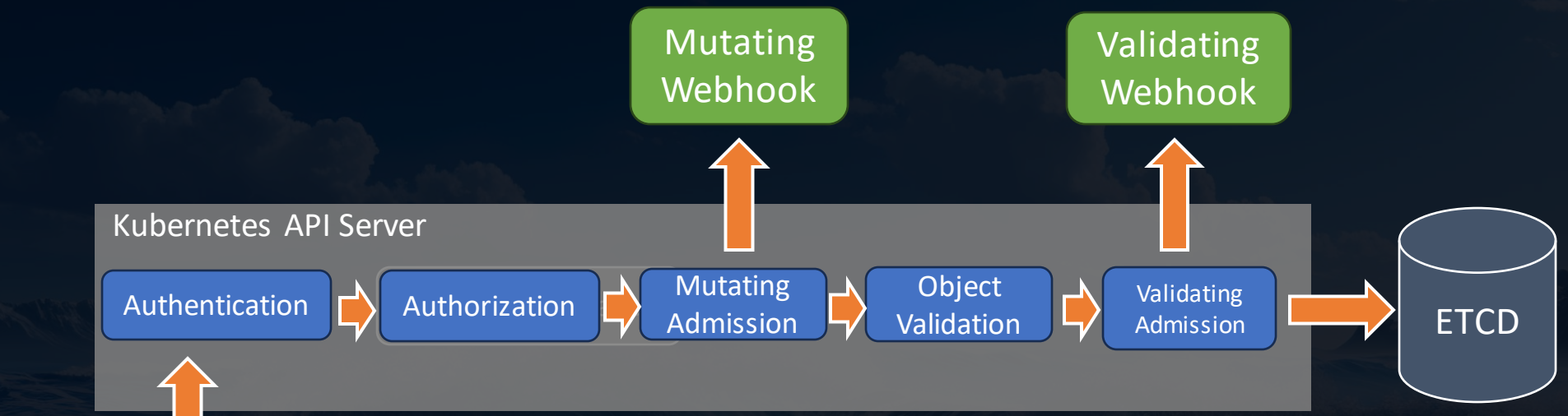


Señor **Developer / Operator**



- Executes the mutation by sending requests to webhook server
- Matching webhooks are called in serial
- Example use cases:
  - injecting side cars (Service Mesh)
  - Add values to Pods without limit/request set





Señor **Developer / Operator**



- Allows you to intercept and validate requests
- Can be run in parallel, as they don't mutate objects
- Example use case: restricting resource creation like DaemonSets


Mutating
Neat, how does the implementation of a Admission Webhook look like?
Allows you to intercept and validate requests
Can be run in parallel, as they don't mutate objects
Example use case: restricting resource creation like DeamonSets

```
139         deployment := appsv1.Deployment{}
140         if _, _, err := deserializer.Decode(raw, nil, &deployment); err != nil {
141                 log.Err(err)
142                 return &admission.AdmissionResponse{
143                         Result: &metav1.Status{
144                                 Message: err.Error(),
145                         },
146                 }
147         }
148         if !strings.HasPrefix(deployment.GetName(), "prod-") {
149                 return &admission.AdmissionResponse{
150                         Allowed: false, Result: &metav1.Status{
151                                 Message: "Deployment's prefix name \"prod\" not found",
152                         },
153                 }
154         }
155         return &admission.AdmissionResponse{Allowed: true}
156 }
157
158 func main() {
159         var tlsKey, tlsCert string
160         flag.StringVar(&tlsKey, "tlsKey", "/etc/certs/tls.key", "Path to the TLS key")
161         flag.StringVar(&tlsCert, "tlsCert", "/etc/certs/tls.crt", "Path to the TLS cert
162         flag.Parse()
163         http.HandleFunc("/mutate", serveMutate)
164         http.HandleFunc("/validate", serveValidate)
165         log.Info().Msg("Server started ...")
166         log.Fatal().Err(http.ListenAndServeTLS(":8443", tlsCert, tlsKey, nil)).Msg("web
167 }
```

main.go hosted with ❤ by GitHub view raw

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: webhook-server
5    namespace: production
6    labels:
7      app: webhook-server
8  spec:
9    replicas: 1
10   selector:
11     matchLabels:
12       app: webhook-server
13   template:
14     metadata:
15       labels:
16         app: webhook-server
17     spec:
18       containers:
19         - name: webhook-server
20           image: bashayralabdullah/webhook-server:v1.0
21           imagePullPolicy: Always
22           ports:
23             - containerPort: 8443
24           volumeMounts:
25             - name: tls-certs
26               mountPath: /etc/certs
27               readOnly: true
28       volumes:
29         - name: tls-certs
30           secret:
31             secretName: webhook-server-tls
32 ---
33 apiVersion: v1
34 kind: Service
35 metadata:
36   name: webhook-server
37   namespace: production
38 spec:
39   selector:
40     app: webhook-server
41   ports:
42     - port: 443
43       targetPort: 8443
```

webhook_server.yml hosted with ❤ by GitHub view raw

Source Medium – Build your own Admission Webhook

```
139         deployment := appsv1.Deployment{}
140         if _, _, err := deserializer.Decode(
141                 log.Err(err
142
143
144
145
146
147
148         if !
149                 return &admission.AdmissionResp
150                         Allowed: false, Result: &me
151                                 Message: "Deployment's
152                         },
153                 }
154         }
155         return &admission.AdmissionResponse{Allowed: true}
156 }
157
158 func main() {
159         var tlsKey, tlsCert string
160         flag.StringVar(&tlsKey, "tlsKey", "/etc/certs/tls.key", "Pa
161         flag.StringVar(&tlsCert, "tlsCert", "/etc/certs/tls.crt", "
162         flag.Parse()
163         http.HandleFunc("/mutate", serveMutate)
164         http.HandleFunc("/validate", serveValidate)
165         log.Info().Msg("Server started ...")
166         log.Fatal().Err(http.ListenAndServeTLS(":8443", tlsCert, t
167 }
```

main.go hosted with ❤ by GitHub view raw

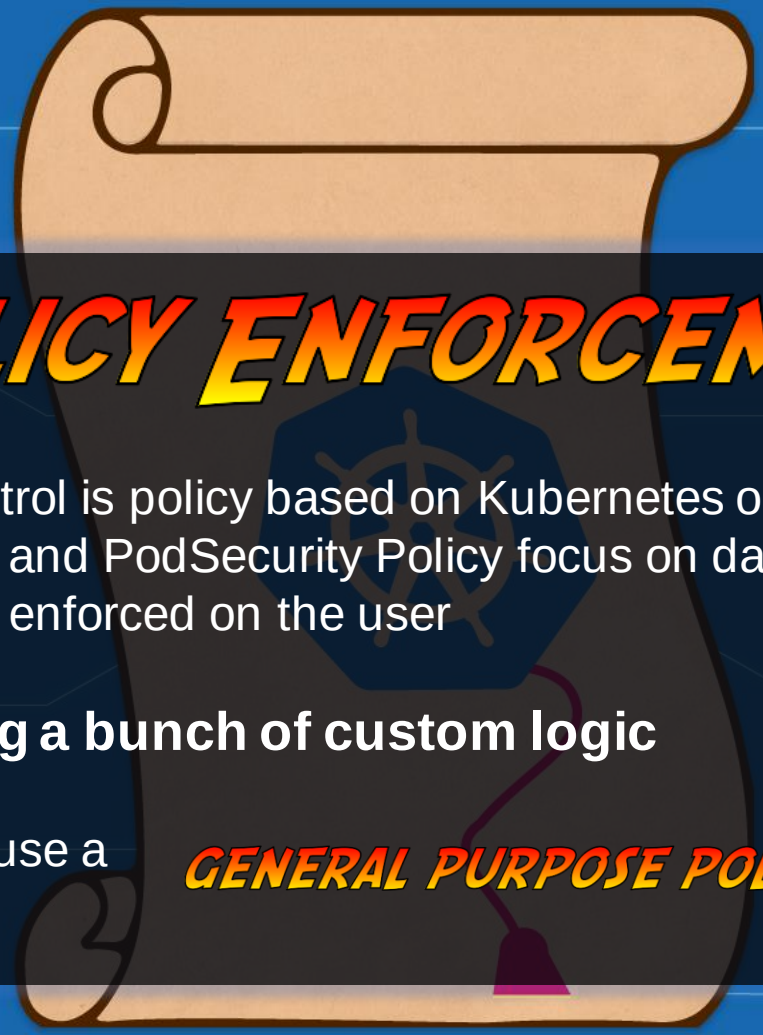
```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: webhook-server
5    namespace: production
6    labels:
7      app: webhook-server
8  spec:
9    replicas: 1
10   selector:
11     matchLabels:
```

```
42     - port: 443
43       targetPort: 8443
```

webhook_server.yml hosted with ❤ by GitHub view raw



Source: Medium – Build your own Admission Webhook



POLICY ENFORCEMENT

- Admission Control is policy based on Kubernetes objects
- Network Policy and PodSecurity Policy focus on data plane policy
- RBAC is policy enforced on the user

To avoid writing a bunch of custom logic

➔ Let‘s use a **GENERAL PURPOSE POLICY ENGINE**



Where can I get a „Policy Engine“ ?

POLICY ENFORCEMENT

- Admission Control is policy based on Kubernetes objects
- Network Policy and PodSecurity Policy focus on data plane policy
- RBAC is policy enforced on the user

To avoid writing a bunch of custom logic

➔ Let‘s use a GENERAL PURPOSE POLICY ENGINE



Where can I get a „Policy Engine“ ?

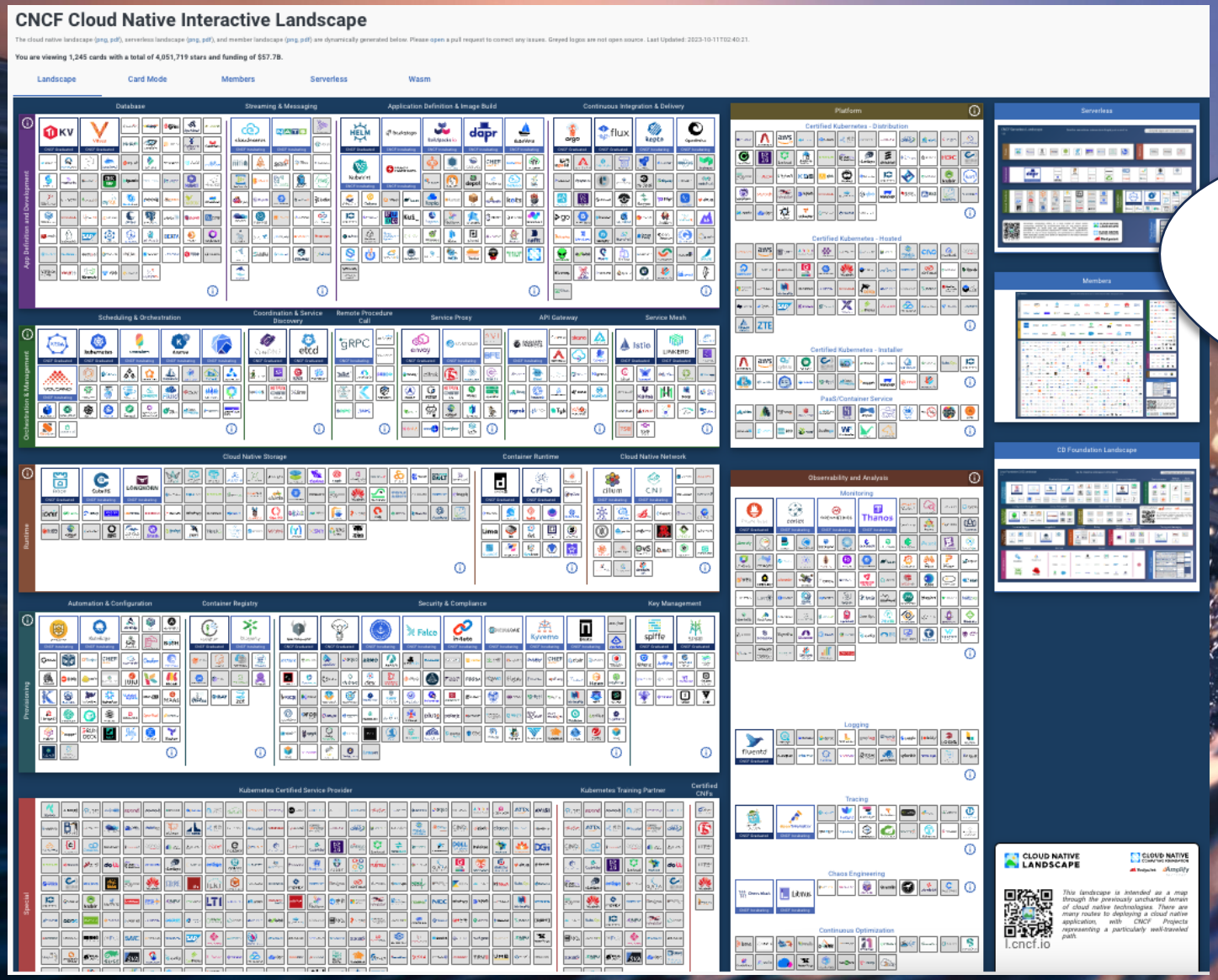
POLICY ENFORCEMENT

- Admission Control is policy based on Kubernetes objects
- Networ… policy
- RBAC

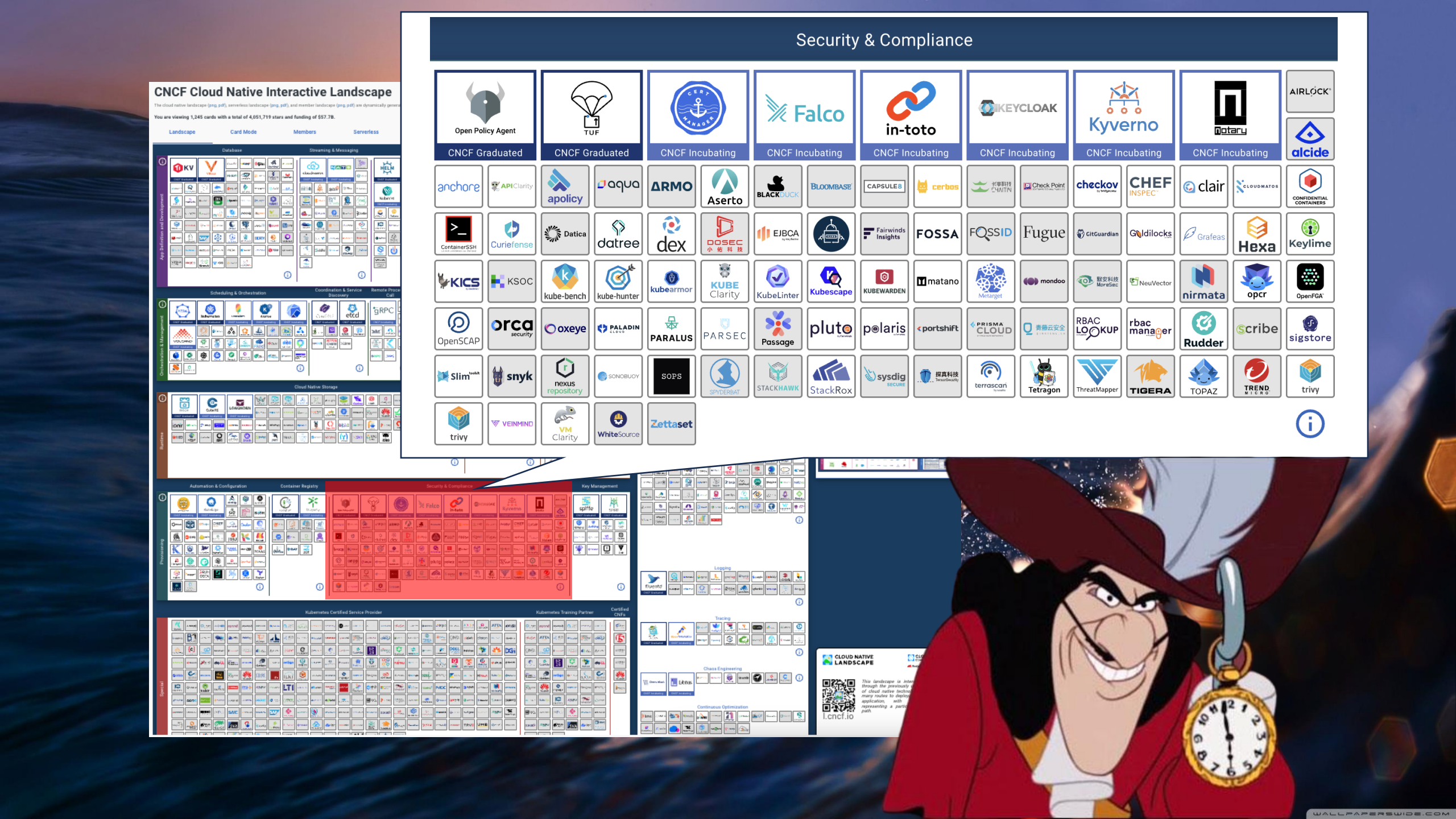
To avoi…

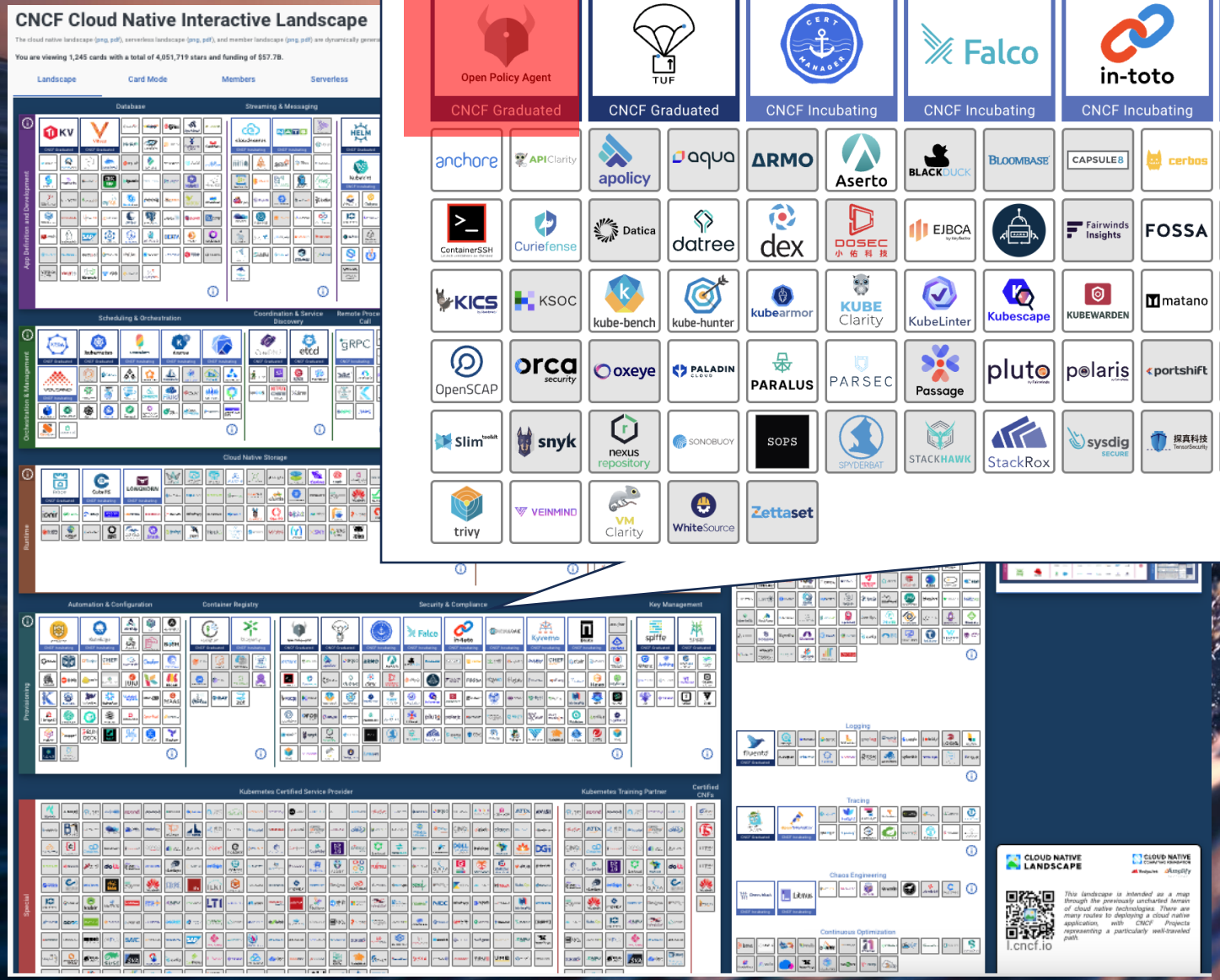
CLOUD NATIVE LANDSCAPE

➔ Let‘s use a GENERAL PURPOSE POLICY ENGINE



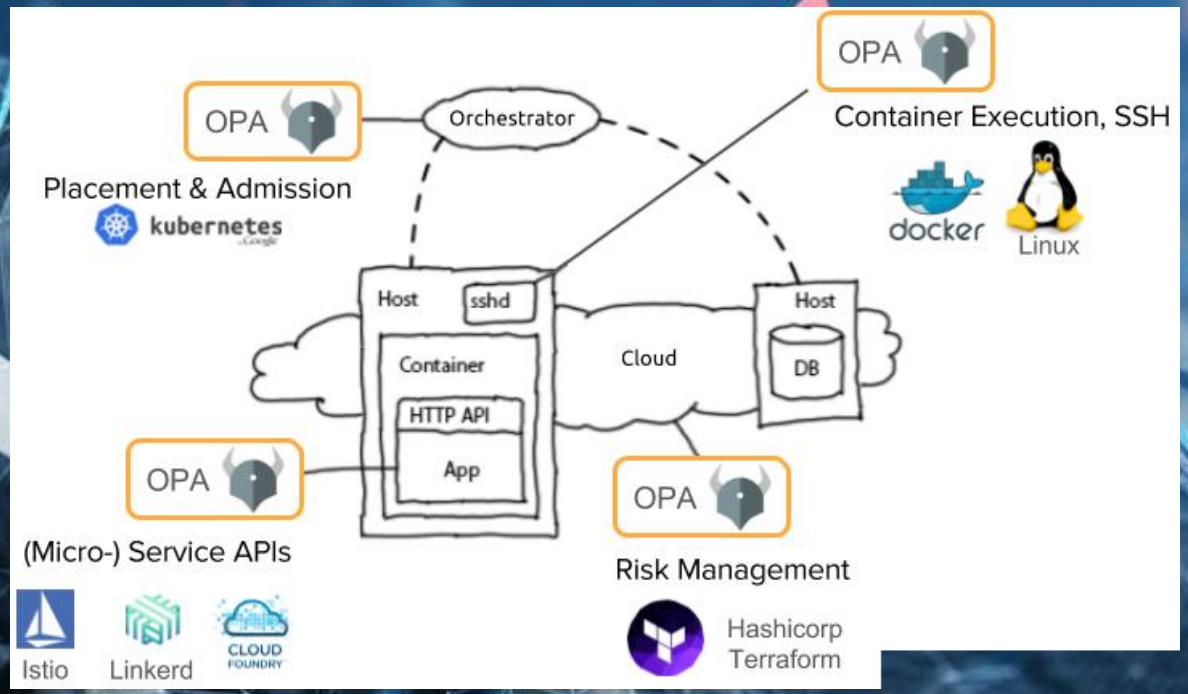
This is a pixel of DOOM, where is it?


CNCF Cloud Native Interactive Landscape
Security & Compliance
CNCF Graduated
CNCF Graduated
CNCF Incubating
CNCF Incubating
CNCF Incubating
CNCF Incubating
CNCF Incubating
CNCF Incubating
Open Policy Agent
TUF
Falco
in-toto
KEYCLOAK
Kyverno
Notary
AIRLOCK
alcide
anchore
APIClarity
apolicy
aqua
ARMO
Aserto
BLACKDUCK
BLOOMBASE
CAPSULE8
cerbos
CHAITIN
Check Point
checkov
CHEF INSPEC
clair
CLOUDMATOS
CONFIDENTIAL CONTAINERS
ContainerSSH
Curiefense
Datica
datree
dex
DOSEC
EJBCA
Fairwinds Insights
FOSSA
FOSSID
Fugue
GitGuardian
Goldilocks
Grafeas
Hexa
Keylime
KICS
KSOC
kube-bench
kube-hunter
kubearmor
KUBE Clarity
KubeLinter
Kubescape
KUBEWARDEN
matano
Metarget
mondoo
NeuVector
nirmata
opcr
OpenFGA
OpenSCAP
orca security
oxeye
PALADIN CLOUD
PARALUS
PARSEC
Passage
pluto
polaris
portshift
PRISMA CLOUD
RBAC LOOKUP
rbac manager
Rudder
scribe
sigstore
Slim
snyk
nexus repository
SONOBUOY
SOPS
SPYDERBAT
STACKHAWK
StackRox
sysdig SECURE
terrascan
Tetragon
ThreatMapper
TIGERA
TOPAZ
TREND MICRO
trivy
VEINMIND
VM Clarity
WhiteSource
Zettaset
trivy
Database
Streaming & Messaging
Scheduling & Orchestration
Coordination & Service Discovery
Remote Procedure Call
Cloud Native Storage
Automation & Configuration
Container Registry
Security & Compliance
Key Management
Logging
Tracing
Chaos Engineering
Continuous Optimization
Kubernetes Certified Service Provider
Kubernetes Training Partner
Certified CNFs
CLOUD NATIVE LANDSCAPE
i.cncf.io



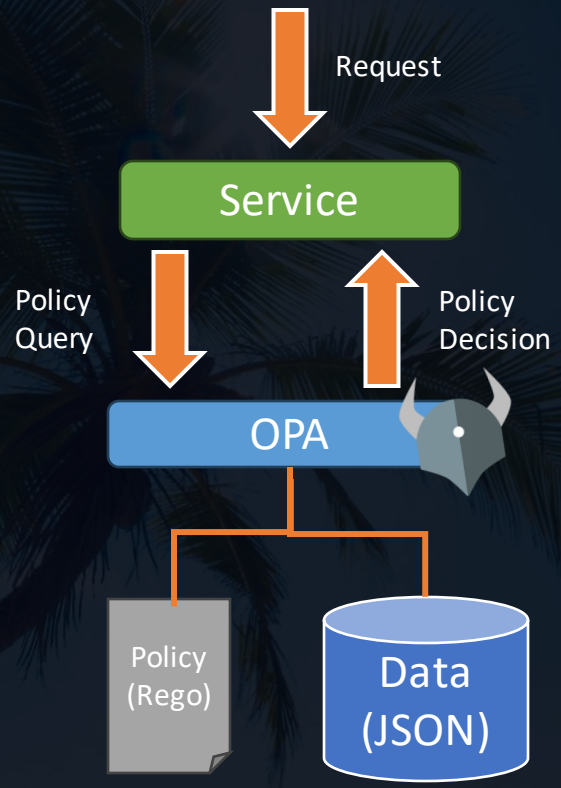
WALLPAPERSWIDE.COM

OPEN POLICY AGENT



- CNCF Graduated Project
- General purpose policy engine
- Can be used across the stack
  ➔ Claims by it self to be a full stack solution
- Loved by Security Nerds
- Uses a Declarative policy language (Rego)

Source: https://blog.openpolicyagent.org/opas-full-stack-policy-language-caeaadb1e077



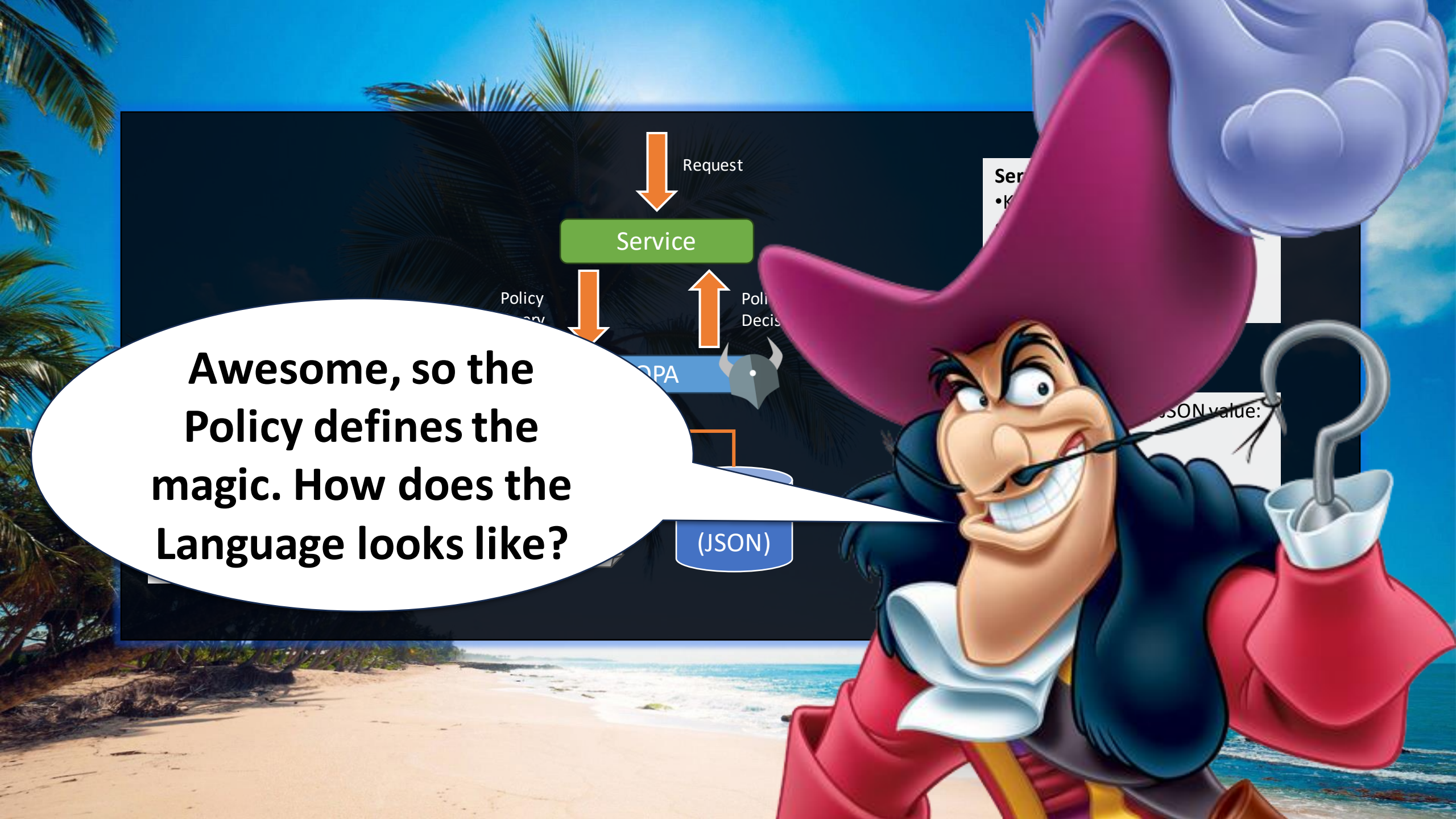
Service refers to:
- Kubernetes API
- Custom API
- SSH Daemon
- Terraform
- Authorization API

Input can be any JSON value:

```
"kind": "Service",
   "metadata": {
       "annotations": {
           env: dev
       }
```

Output can be any JSON value:

```
”true“,
    ”request annotated”,
  “annotations": {
       networkEnv: dev-int
}
```


Request
Service
Policy
Poli
Decis
OPA
(JSON)
Awesome, so the Policy defines the magic. How does the Language looks like?

REGO EXAMPLE

```
package example

import future.keywords      # we want "every" and "in"

default allow := true {     # allow is true if...
  count(violation) == 0     # there are zero violations.
}

violation[server.id] {                  # a server is in the violation set if...
  some server in public_servers         # it exists in the 'public_servers' set and...
  "http" in server.protocols            # it contains the insecure "http" protocol.
}

public_servers[server] {                # a server exists in the public_servers set if...
        some server in input.servers    # it exists in the input.servers collection and...
        some port in server.ports       # it references a port in input.ports collection and...
        some input_port in input.ports
        port == input_port.id
        some input_network in input.networks   # the port references a network in the input.networks collection and...
        input_port.network == input_network.id
        input_network.public            # the network is public.
}
```

```
package example

import future.keywords      # we want "every" and "in"

default allow := true {     # allow is true if...
  count(violation) == 0     # there are zero violations.
}

violation[server.id] {                  # a server is in the violation set if...
  some server in public_servers         # it exists in the 'public_servers' set and...
  "http" in server.protocols            # it contains the insecure "http" protocol.
}

public_servers[server] {                # a server exists in the public_servers set if...
          some server in input.servers          # it exists in the input.servers collection and...
          some port in server.ports             # it references a port in input.ports collection and...
          some input_port in input.ports
          port == input_port.id
          some input_network in input.networks   # the port references a network in the input.networks collecti
          input_port.network == input_network.id
          input_network.public                  # the network is public.
}
```

mY bRaIn huRtS! Isn‘t this the definition of arcane tribal policy knowledge ?



```
package example

import future.keywords      # we want "every" and "in"

default allow := true {     # allow
  count(violation) == 0     # there
}

violation[server.id] {                #
  some server in public_servers
  "http" in server.protocols
}

public_servers[server] {
          some server in input.ser
          some port in server.port
          some input_port in inpu
          port == input_port.id
          some input_network in i
          input_port.network == i
          input_network.public              # the network is public.
}
```



mY bRaIn huRtS! Isn‘t this the definition of arcane tribal policy knowledge ?



MANAGEMENT OF POLICIES



Developer

YAML

Deploys App

YAML

Build, maintain and harden the cluster



Platform Operator

YAML

Audits Plattform



Divine Governance & Compliance God Instance

MANAGEMENT OF POLICIES



Developer

YAML

Deploys App



YAML

Audits Plattform



Divine Governance & Compliance
God Instance

YAML

Build, maintain and harden
the cluster



Platform Operator



REGO POLICY

Creates and maintain policies



Security Team

MANAGEMENT OF POLICIES



Developer

YAML

Deploys App



YAML

Audits Plattform



Divine Governance & Compliance
God Instance

YAML

Build, maintain and harden
the cluster



Platform Operator



REGO POLICY

Creates and maintain policies

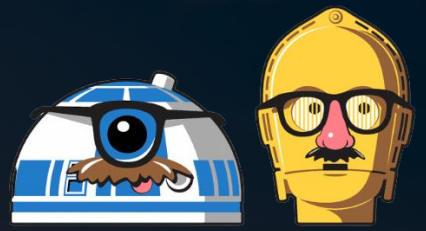


Security Team

Management of Policies

Potential issues:

- Rego has a high learning curve
- Everyone is Working in YAML
  - ➔ Rego as a language is decoupled from this ecosystem
- Security Team owns the arcane tribal magic about the Policies
  - ➔ How can this scale?
  - ➔ Single Point of Failure, why not trust in swarm intelligence?
- Throwing a Policy over the fence might…
  - ➔ … cause mayham and havoc in workloads
  - ➔ … be decoupled of the needs of the Devs- and Plattform-Teams
  - ➔ … be solved already in a different way



THESE AREN'T THE DROIDS
YOU ARE LOOKING FOR …

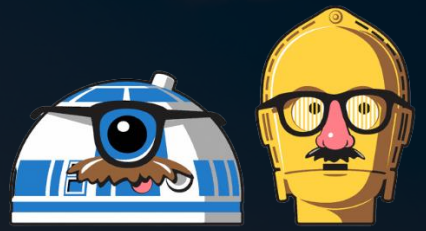
MANAGEMENT OF POLICIES

Potential issues:

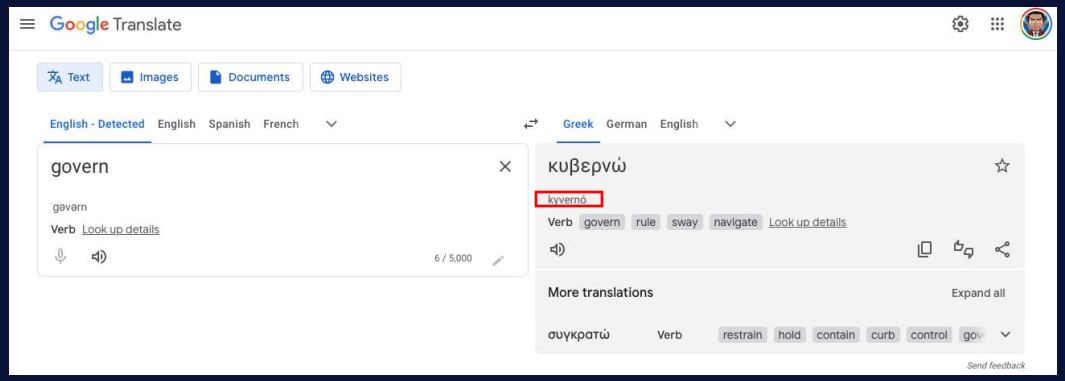
- Rego has a high learning curve
- Everyone is Working in YAML
  - ➔ Rego as a lan[illegible]age is decoupled fr[illegible]system
- Security Team owns t[illegible]ane tribal magic ab[illegible]Policie
  - ➔ How ca[illegible]cale?
  - ➔ [illegible]f Failure, why not trus[illegible]ntelligence?
- Thro[illegible]g a Policy [illegible]r the fence might…
  - caus[illegible]and havoc in worklo[illegible]
  - de[illegible]the needs of the Dev[illegible]orm-Teams
  - [illegible]olved alrea[illegible]in a different way

This is NOT solving my issue …

Have you tried already Kyverno ?



THESE AREN'T THE DROIDS YOU ARE LOOKING FOR …

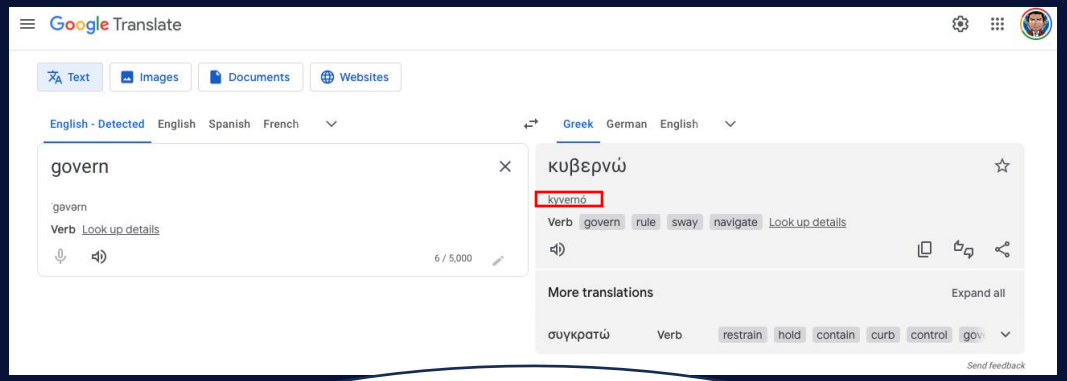


- Uses Kubernetes resources, patterns, idioms
- Deployed as a workload in the Cluster
  ➔ Eat your own Dogfoot
- Pre-generated policy available
  ➔ https://kyverno.io/policies/
- Custom policies written in YAML
- Can be distributed using Helm Charts

- Sadly still incubating project on CNCF
- Has a high chance to graduate soon



Kyverno



Aye, how can I image this also in comparison with OPA?

- Sadly still Incubating Project on CNCF
- Has a high chance to Gradutate soon

EXAMPLE: REQUIRE-NO-ROOTFS

CUSTOM RESOURCE DEFINITION

K8S DEPLOYMENT YAML

REGO POLICY

REGO LIBS

VS

```yaml
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8spspreadonlyrootfilesystem
  annotations:
    metadata.gatekeeper.sh/title: "Read Only Root Filesystem"
    metadata.gatekeeper.sh/version: 1.0.1
spec:
  crd:
    spec:
      names:
        kind: K8sPSPReadOnlyRootFilesystem
      validation:
        # Schema for the `parameters` field
        openAPIV3Schema:
          type: object
          properties:
            exemptImages:
              type: array
              items:
                type: string
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8spspreadonlyrootfilesystem

        import data.lib.exclude_update.is_update
        import data.lib.exempt_container.is_exempt

        violation[{"msg": msg, "details": {}}] {
            # spec.containers.readOnlyRootFilesystem field is immutable.
            not is_update(input.review)

            c := input_containers[_]
            not is_exempt(c)
            input_read_only_root_fs(c)
            msg := sprintf("only read-only root filesystem container is allowed: %v", [c.name])
        }

        input_read_only_root_fs(c) {
            not has_field(c, "securityContext")
        }

        input_read_only_root_fs(c) {
            not c.securityContext.readOnlyRootFilesystem == true
        }

        input_containers[c] {
            c := input.review.object.spec.containers[_]
        }

        input_containers[c] {
            c := input.review.object.spec.initContainers[_]
        }

        input_containers[c] {
            c := input.review.object.spec.ephemeralContainers[_]
        }

        # has_field returns whether an object has a field
        has_field(object, field) = true {
            object[field]
        }
      libs:
        - |
          package lib.exclude_update

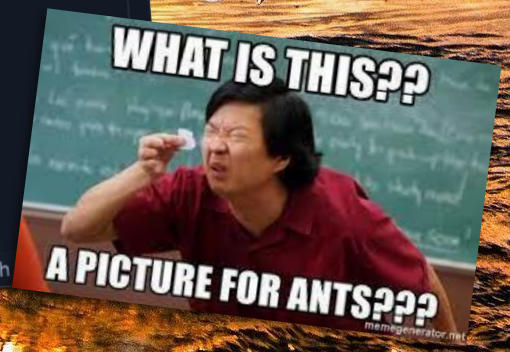
          is_update(review) {
              review.operation == "UPDATE"
          }
        - |
          package lib.exempt_container

          is_exempt(container) {
              exempt_images := object.get(object.get(input, "parameters", {}), "exemptImages", [])
              img := container.image
              exemption := exempt_images[_]
              _matches_exemption(img, exemption)
          }

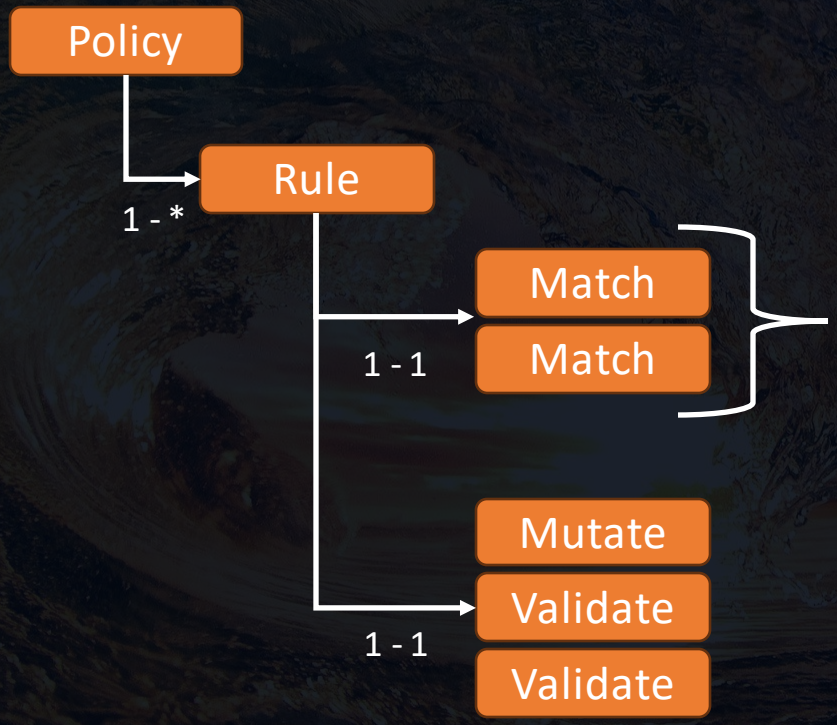
          _matches_exemption(img, exemption) {
              not endswith(exemption, "*")
              exemption == img
```

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: require-ro-rootfs
spec:
  validationFailureAction: audit
  background: true
  rules:
  - name: validate-readOnlyRootFilesystem
    match:
      any:
      - resources:
          kinds:
          - Pod
    validate:
      message: "Root filesystem must be read-only."
      pattern:
        spec:
          containers:
          - securityContext:
              readOnlyRootFilesystem: true
```

Kyverno

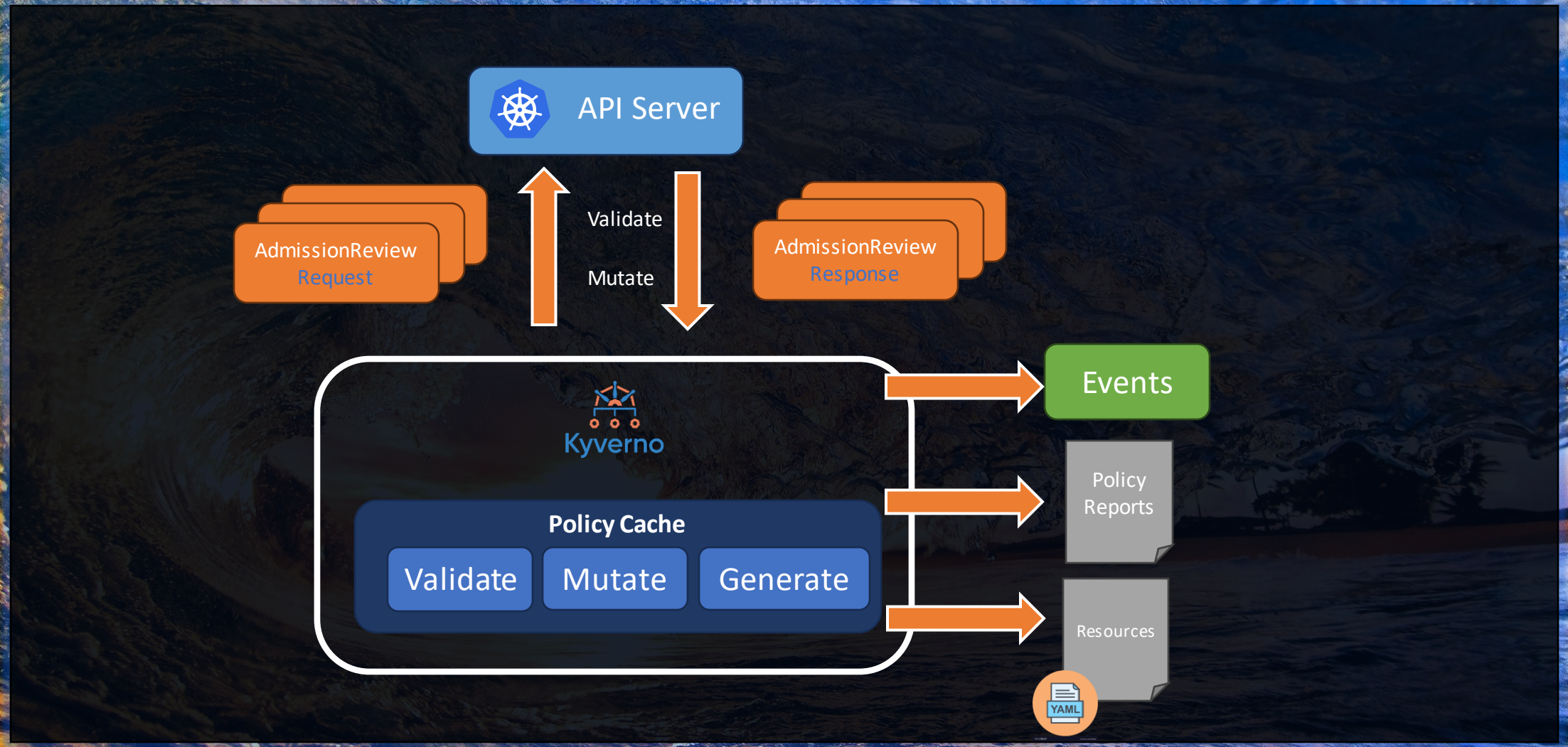


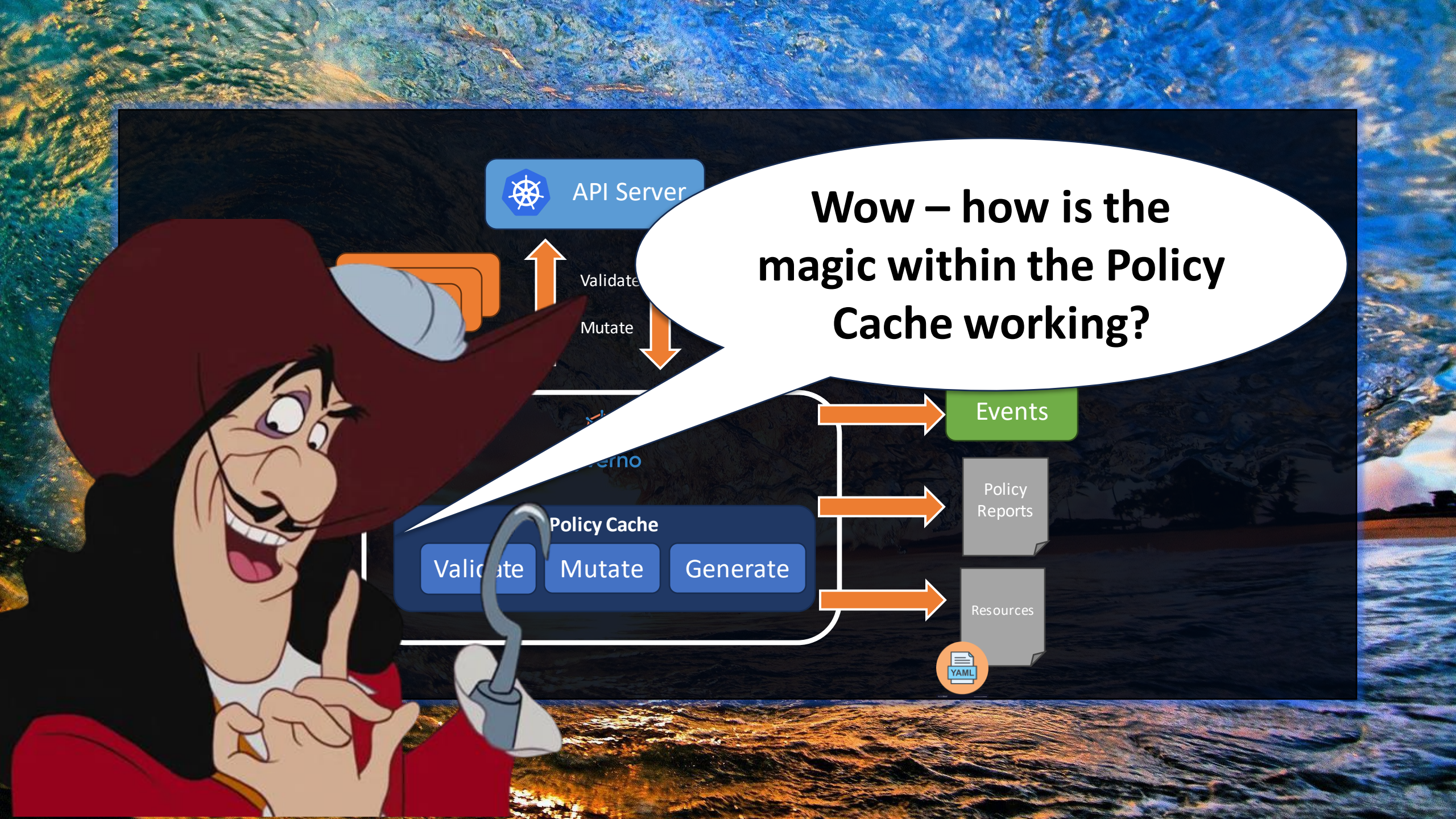
Anatomy of a Kyverno Policy



- Kinds
- Names (with wildcards)
- Label Selectors (with wildcards)
- Namespaces (with wildcards)
- Namespace Selector
- User Roles
- User Groups
- Usernames




API Server
AdmissionReview
Request
Validate
Mutate
AdmissionReview
Response
Kyverno
Policy Cache
Validate
Mutate
Generate
Events
Policy
Reports
Resources
YAML


API Server
Validate
Mutate
Wow – how is the magic within the Policy Cache working?
Events
Policy Reports
Resources
YAML
Policy Cache
Mutate
Generate

MUTATE

- JSON Patch (RFC 6902)
  - Use for precise updates
- StrategicMergePatch
  - Use for describing intent
  - Anchors for conditional logic

```yaml
mutate:
patches:
- path: "/spec/template/spec/mypolicies/0"
op: add
value:
- image: "mypod:1.0.0"
name: "init-secrets"
```

IF-THEN-ELSE

```yaml
mutate:
  overlay:
    subsets:
    - ports:
      - (name): "secure*"
        port: 8443
```

IF-NOT-DEFINED

```yaml
mutate:
  overlay:
    subsets:
    - ports:
      +(port): 8443
```



Source: https://kyverno.io/docs/writing-policies/mutate/

VALIDATE

- Overlays with patterns specify dersired state
- Matches all defined fields
- Patterns for aplphanumeric characters
  - * = zero or more
  - ? = any one
- Operators
  - >, <, >=, <=,
  - | (logical or), , &(logical and)
  - - (within a range), !-(outside a range)

```yaml
validate:
  message: "Label env is required"
  pattern:
    spec:
      template:
        metadata:
          labels:
            env: "?"
```

Source: https://kyverno.io/docs/writing-policies/validate/



GENERATE

- Triggers when a new resource is created
- Useful in creating defaults for a namespace like
  - NetworkPolicies
  - Secrets (clone/sync)
  - ConfigMaps
  - ResourceQuota
  - Kubernetes Events
  - …
- Clones existing resources or copies in-line data
- Can optionally keep data

```
generate:
  kind: NetworkPolicy
  name: deny-all-traffic
  data:
    spec:
    podSelector:
      matchLabels: {}
      matchExpressions: []
    policyTypes: []
    metadata:
      labels:
        policyname: "default"
```

Source: https://kyverno.io/docs/writing-policies/generate/



GENERATE

- Triggers when a new resource is created
- Useful in creating defaults for a namespace like
  - Net
- Ca

Awesome – now let put everything together in an example

```
genera
  kind:
  name:
  data:
    spec:
    podSelector:
      matchLabels: {}
      matchExpressions:
    policyTypes: []

        policyname
```

Source: https://kyverno.io/docs/writing-policies/generate/



Example: Tackle Image Rotting with continuous Trivy Vulnerability Scans

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: check-vulnerabilities
spec:
  validationFailureAction: enforce
  webhookTimeoutSeconds: 10
  failurePolicy: Fail
  rules:
    - name: not-older-than-one-week
      match:
        any:
        - resources:
            kinds:
              - Pod
      verifyImages:
      - imageReferences:
        - "CONTAINER-REGISTRY/*:*"
        attestations:
        - predicateType: cosign.sigstore.dev/attestation/vuln/v1
          conditions:
          - all:
            - key: "{{ time_since('','{{metadata.scanFinishedOn}}','') }}"
              operator: LessThanOrEquals
              value: "168h"
```

What it does:

- Scans with Trivy (Vulnerability Scanner)
- Triggers only on images older then 168h
- Uses Cosgin to attest „health“ of the image

What do you get:

- Makes rotting Images transparent
- Continous status instead of only once in you CI/CD Pipeline

Source: https://aquasecurity.github.io/trivy/v0.32/tutorials/kubernetes/kyverno/

Let's wrap it up ...

- Kyverno might be a good solution for Governance in Kubernetes
- Don‘t decouple Dev/Plattform Guys from the Security Guys
  ➔ Let‘s work together!
- The best Security Tool won‘t work if it‘s not fitting within the landscape

Now your turn

- What policies do you use?
- Are you using even at all a Policy Engine?

